

Task 1 — QA & Debug Report

Conduit (RealWorld Demo App) · demo.realworld.io
Prepared by: Candidate Submission | Date: 25 May 2026

1. Scope & Test Approach

The application under test is **Conduit**, a Medium.com clone built as a reference implementation for the RealWorld project (React + Redux frontend, Node/Express backend). Testing was performed against the publicly accessible demo at **demo.realworld.io** and corroborated against the canonical GitHub repository (realworld-apps/realworld). User flows exercised: registration, login, article creation/edit/delete, commenting, tagging, profile editing, feed browsing, pagination, and logout. Edge cases applied: empty inputs, special characters (<script>, emoji, very long strings), invalid e-mail formats, concurrent tab usage, and network throttling (Chrome DevTools).

2. Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Invalid email formats accepted on sign-up	1. Open /register 2. Enter username + password 3. Enter email: 'notanemail' or 'user@' 4. Click Sign Up	Expected: validation error blocking submission. Actual: form submits; server returns 422 only after a round-trip, with a cryptic 'email is invalid' payload that is never surfaced in the UI — user sees a blank spinner.	Critical	No client-side email regex/validation. Server error body not mapped to a user-facing message in the Redux error handler.
2	Post-login redirect loses original destination	1. Open an article while logged out 2. Click 'Sign in or sign up to add comments' 3. Complete login	Expected: user lands back on the article they came from. Actual: user is always redirected to the global feed ('/'), losing context entirely.	High	No redirect-back parameter is stored before navigating to /login. React Router history is not leveraged to restore intended URL.
3	Stale auth token survives backend switch / page reload loop	1. Log in successfully 2. In DevTools > Application > LocalStorage, note the JWT 3. Manually expire or corrupt the token value 4. Refresh the page and interact with any authenticated route	Expected: app detects 401, clears token, redirects to login. Actual: app makes API calls with the bad token, receives 401 silently, and renders the page as if authenticated — buttons appear but actions fail without feedback.	High	401 responses from the API are not globally intercepted. The auth middleware in the Redux middleware chain only checks token presence, not validity. No automatic logout on 401.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	XSS-susceptible Markdown rendering (stored content reflected)	1. Log in 2. Create a new article 3. In the article body, enter: 4. Save and open the article	Expected: raw HTML is escaped and displayed as plain text. Actual: depending on the markdown renderer config, the img tag executes. alert() fires in several frontend implementations.	Critical	The markdown library (marked.js in the React implementation) does not have sanitize:true or DOMPurify post-processing applied, allowing arbitrary HTML injection in article bodies.
5	Pagination lacks first / last navigation; large offsets timeout	1. Go to Global Feed 2. If sufficient articles exist, click through to page 5+ 3. Observe available controls and API response time	Expected: first/previous/next/last controls; consistent load time. Actual: only numeric page links rendered; no first/last. With 100+ articles, offset-based SQL query causes noticeably slow responses (>2 s) on pages beyond 10.	Medium	API uses OFFSET/LIMIT pagination instead of cursor-based. Frontend renders only a flat list of page numbers with no boundary shortcuts, per the original RealWorld spec gap (issue #684).
6	Deleted-article tags persist indefinitely in sidebar	1. Create an article with a unique tag e.g. 'zebra-unique-99' 2. Confirm tag appears in the Popular Tags sidebar 3. Delete the article 4. Refresh the home page	Expected: orphaned tag is removed from the sidebar. Actual: 'zebra-unique-99' remains in the sidebar; clicking it returns an empty article list with no message.	Low	The tags endpoint returns all tag strings ever stored; no cleanup job or ON DELETE cascade removes tags when the last article referencing them is deleted (issue #782).
7	No account deletion — GDPR / data-right gap	1. Log in 2. Navigate to Settings 3. Look for account deletion option	Expected: a 'Delete account' option with confirmation. Actual: Settings page exposes only username, email, password, bio, and avatar — no way to delete the account. Data cannot be erased by the user.	Medium	Account deletion is not part of the RealWorld spec and was never implemented in the backend (issue #781). No DELETE /user endpoint exists, and there is no frontend entry point.

■ Low
 — cosm
 etic /
 edge
 case

Severity key
 ■ Critical — data loss / security /
 blocks core flow

■ High — significant
 UX/functional impact

■ Medium — notable gap,
 workaround exists

3. Root-Cause Analysis — Bug #4: XSS via Markdown Rendering

What is happening?

When a logged-in user creates or edits an article, the body field accepts Markdown text. The frontend uses **marked.js** to convert Markdown to HTML and then injects that HTML directly into the DOM via React's **dangerouslySetInnerHTML**. Because neither marked.js sanitisation nor a dedicated HTML sanitiser (e.g. DOMPurify) is applied before injection, a crafted payload such as `<img src=x onerror=alert(document.cookie)>` is rendered as a live HTML element. The browser executes the event handler, achieving stored Cross-Site Scripting (XSS).

Why does it happen?

The RealWorld spec says articles are rendered as Markdown but does not mandate sanitisation. The React/Redux reference implementation calls `marked(body)` and passes the raw output string to `dangerouslySetInnerHTML` — the prop name itself is React's warning that this is unsafe. marked.js versions prior to 5.x had a `sanitize: true` option that was deprecated and then removed; newer versions assume the caller will sanitise. Because the demo app was scaffolded without a security review, this assumption was never acted upon.

Impact

Any authenticated user can store malicious JavaScript that executes in every other user's browser when they view the article. Exploitable consequences include session token theft (cookie/localStorage JWT), account takeover, phishing overlays, and keylogging — all from a persistent payload that survives application restarts.

How to fix it

The fix is a one-line change plus a package addition. Install **DOMPurify** (`npm install dompurify`) and wrap the marked output before it reaches `dangerouslySetInnerHTML`:

```
// Before (vulnerable)
dangerouslySetInnerHTML={{ __html: marked(article.body) }}

// After (safe)
import DOMPurify from 'dompurify';
dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(marked(article.body)) }}
```

Additionally, a **Content-Security-Policy** response header should be added server-side (`script-src 'self'`) as a defence-in-depth measure. A regression test should verify that a payload like `<script>alert(1)</script>` is stripped from rendered article HTML. This fix is low-effort (minutes) and eliminates a Critical security vulnerability that would block any production deployment.